浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

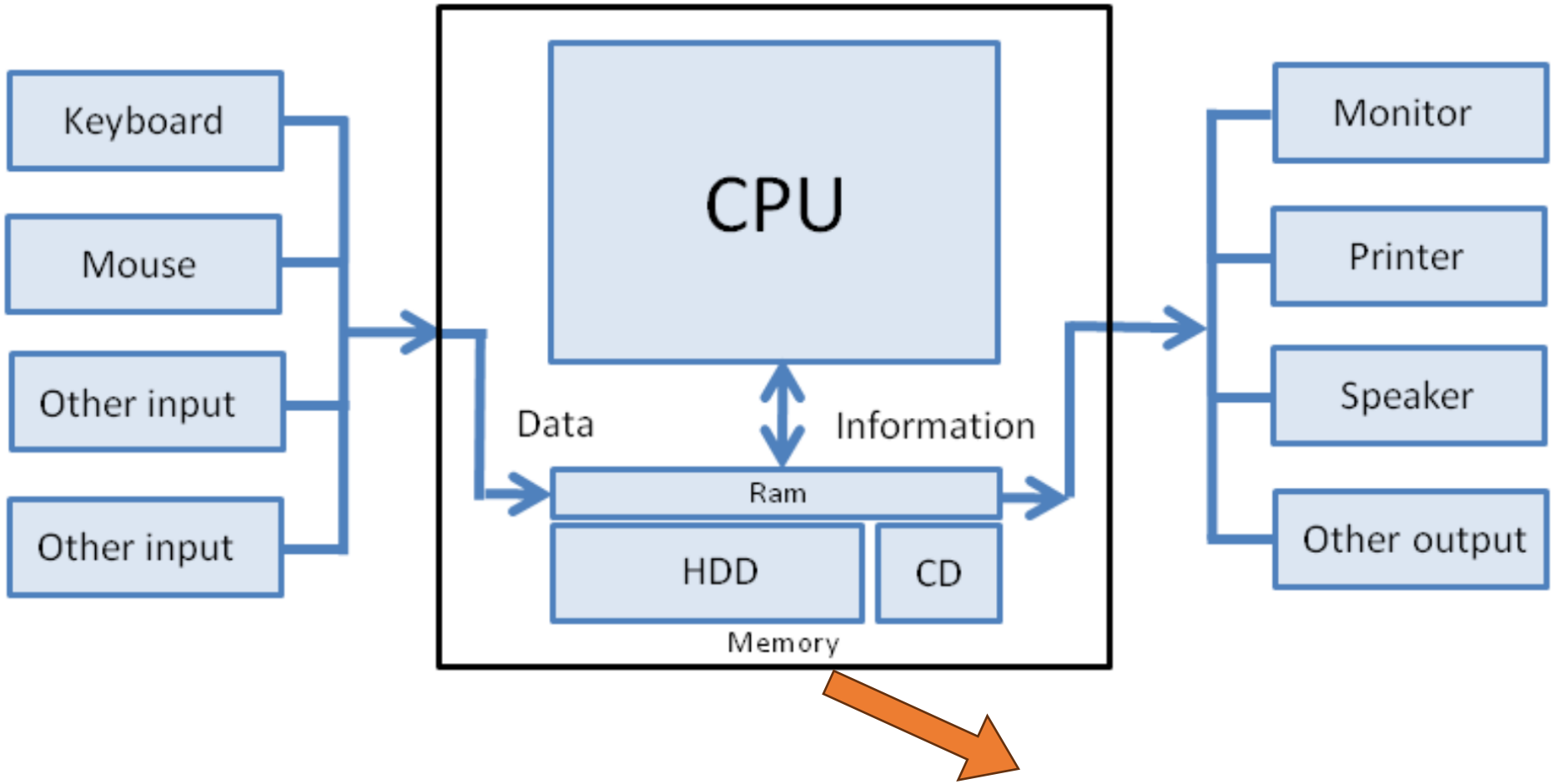
Python III: File Operations

IBI 1, Lecture 7.2

Dr Duncan MacGregor –
duncan.macgregor@ed.ac.uk

Semester 2, 2025/26

Input and Output



Files, files, files

Learning objectives

After this lecture, you should be able to:

- Understand stdin and stdout
- Read to and write from files

This lecture is (another) Python reference guide

"They're encyclopedias, Timmy. . . they're an early form of

- The information here will help you write code, but it's best to practice – not to memorise.
- You can often achieve the same thing by different means, and you should choose what works best for you (not me!).
- You may find solutions that are not presented here, this is not a comprehensive list.

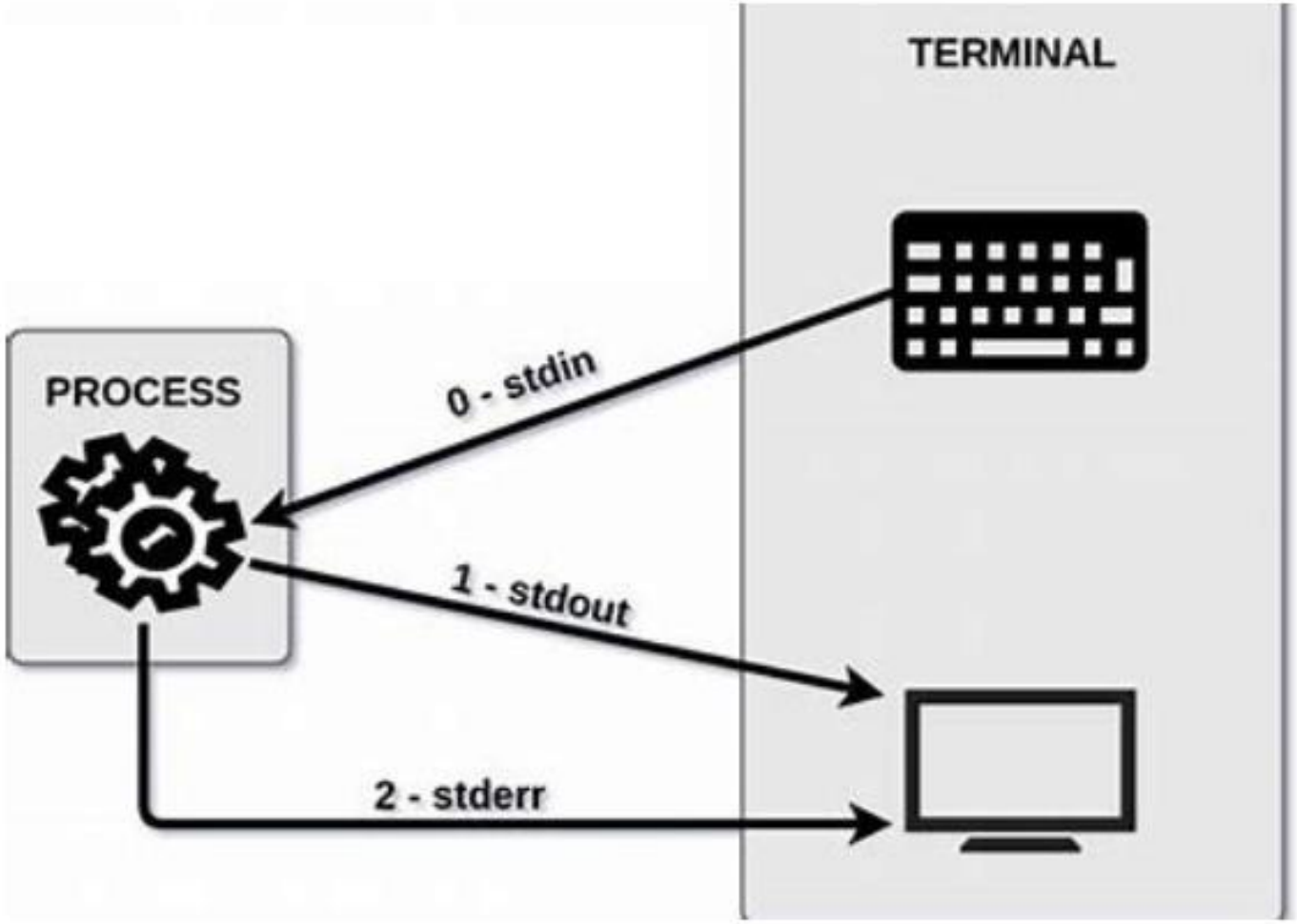
Outline

1. Standard input and standard output
2. File operations
3. pandas

stdin and stdout

- `stdin` is a stream that represents input into a program
- `stdout` is where all your output goes
- `stderr` contains all the error messages (looks a lot like stdout)

```
>>> import sys
```



Read from stdin and print to stdout

- input() waits and reads input from stdin - everything is read as a string
- print() prints the output to stdout

```
>>> a = input()
I like IBI1!
>>> print(a)
I like IBI1!
>>> type (a)
<class 'str'>
```

stdout *vs.* stderr

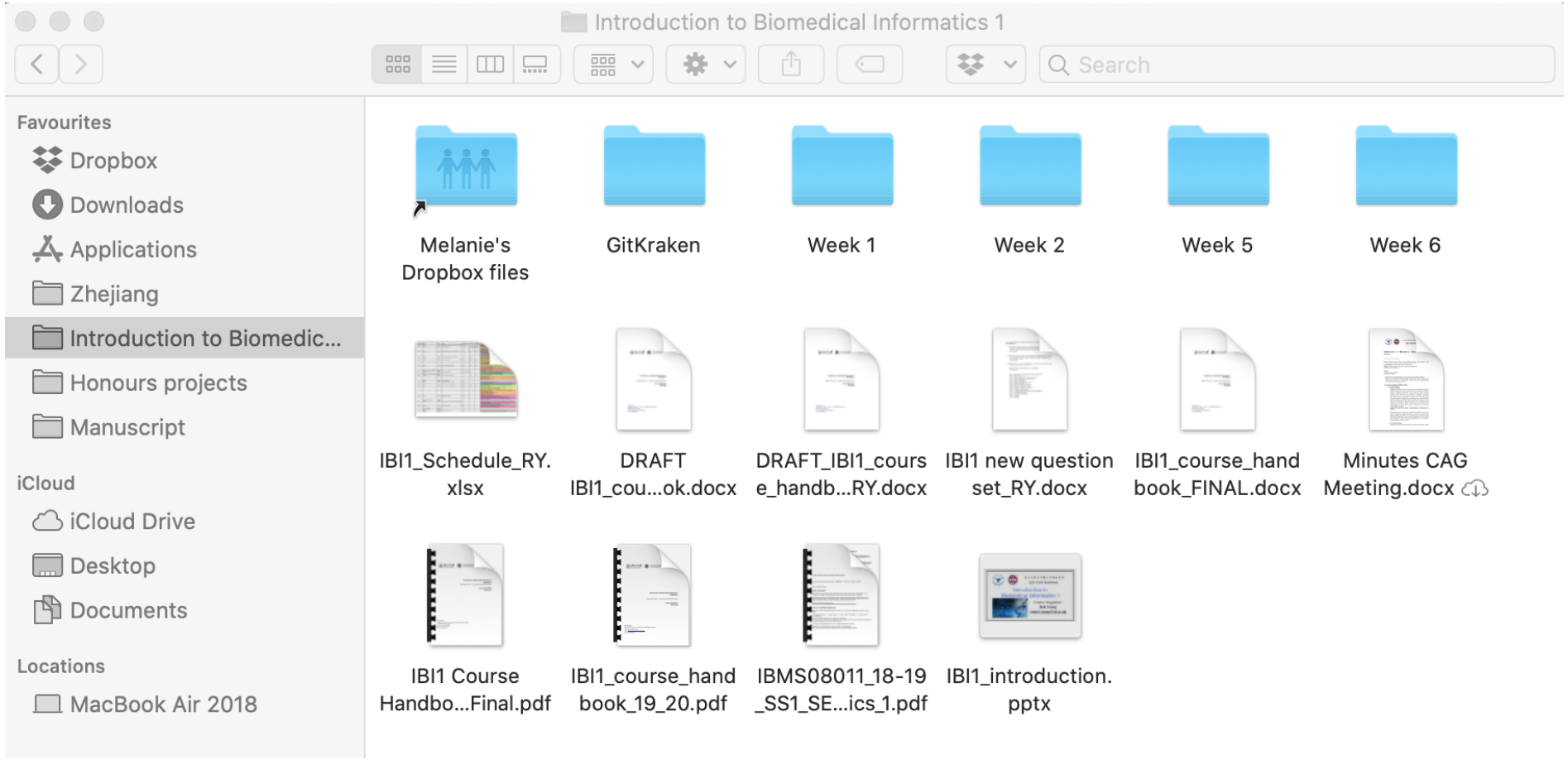
- print() prints the output to stdout
- Error messages are kept in a separate stream known as the stderr

```
>>> a = "four"
>>> b = 2
>>> a + b
TypeError: can only concatenate str (not "int") to str
```

Outline

1. Standard input and standard output
2. File operations
3. pandas

What is a file?



Opening a File

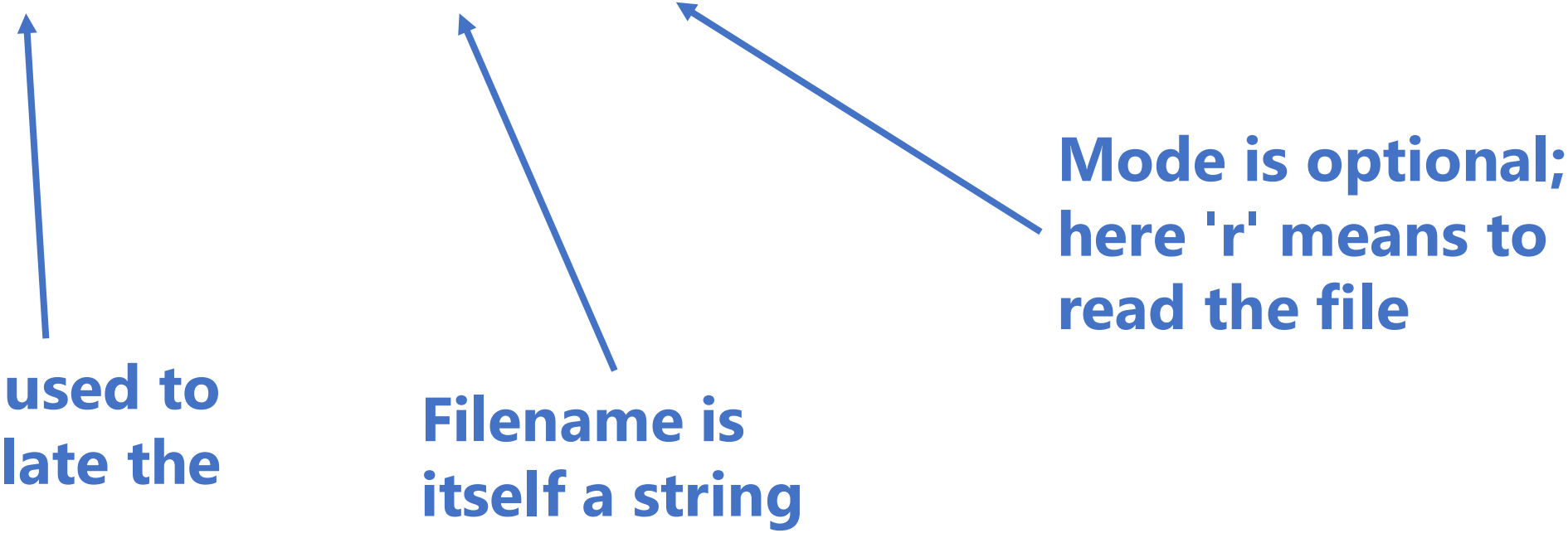
- Before reading a file, we must tell Python which file we are going to work with **and** what we will be doing with the file
- This is done with the **open()** function

- **open()** returns what we call a "**file handle**" – a variable used to perform operations

Using open()

- file_handle = open(filename, mode)

```
>>> input = open('mbox.txt', 'r')
```



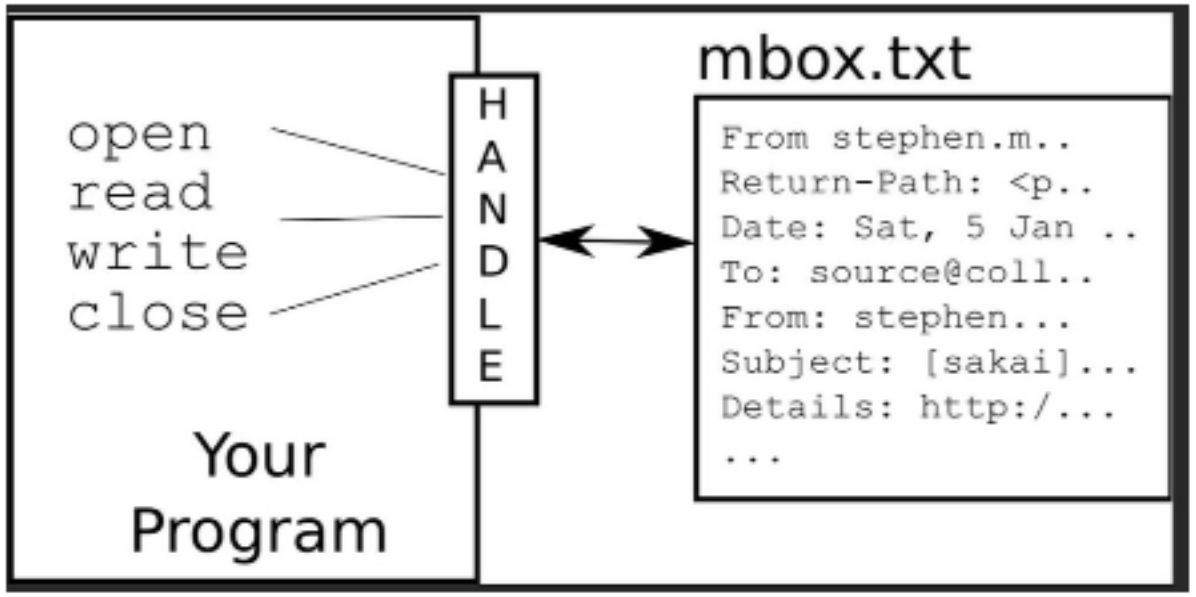
File handle models

Character	Meaning
'r'	open for reading (default)
'w'	open for writing (overwrite the file)
'a'	open for writing (appending if the file exists)
'b'	binary mode
't'	text mode (default)
'x'	open for exclusive writing (fails if the file exists)
'+'	open for updating (reading and writing)

- Note: the default mode is 'r' (open for reading text, a synonym of 'rt')

What does a handle look like?

```
>>> input = open('mbox.txt', 'r')
>>> print(input)
<_io.TextIOWrapper name='mbox.txt' mode = 'r' encoding='UTF-8'>
```



The new line character

- The special character called the "**newline**" indicates when a line ends
- Represented as `\n` in strings – but is still one character

```
>>> print("Hello world")
Hello world

>>> print("Hello\nworld")
Hello
world
```

File progressing

- A text file has **newlines** at the end of each line…or
- A text file can be thought of as a sequence of lines

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008\n
Return-Path: <postmaster@collab.sakaiproject.org\n
Date: Sat, 5 Jan 2008 09:12:18 -0500\n
To: source@collab.sakaiproject.org\n
From: stephen.marquard@uct.ac.za\n
Subject: [sakai] svn commit: r39772 - content/branches/\n
\n
Details: http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772\n
```

File Handle as a Sequence

- A **file handle** open to read can be treated as a sequence of strings where each line in the file is a string in the sequence.
- We can use the for loop (remember that?) to iterate through a sequence

```
>>> input = open('mbox.txt', 'r')
>>> for line in input:
      print(line)
```

 # returns all lines in 'mbox.txt'

Count the lines in a file

- Open a **file** for read-only and use a **for** loop to read each line.
- **Count** the lines and print out at the end. We can use the for loop (remember that?) to iterate through a sequence

```
>>> input = open('mbox.txt', 'r')
>>> count = 0
>>> for line in input:
      print(line)
      count += 1
>>> print('Line count:',count)
Line Count: 1909
```

Reading the whole File

- We can read the whole file (newlines and all!) into a single **string**.
- **`f.read(size)`**, which reads some quantity of data and returns it as a string
- Size is an optional numerical argument (I have never seen it use).

```
>>> input = open('mbox.txt', 'r')
>>> inp = input.read()
>>> print (len(inp))
94625
>>> print(inp[:20])
From stephen.marquar
```

Note: f.read() will read the entire file into memory which is a problem if the file is large.

Searching

- We can use an `if` statement inside our `for` loop to only print lines that meet some criteria.

```
>>> input = open('mbox.txt', 'r')
>>> for line in input:
      if re.search('From:',line):
            print(line)
```

```
From: stephen.marquard@uct.ac.za

From: louis@media.berkeley.edu

From: zqian@umich.edu

From: rjlowe@iupui.edu

From: zqian@umich.edu

From: rjlowe@iupui.edu

From: cwen@iupui.edu
```

Remember: the print statement adds a newline to each line!

Searching: fixed

- The whitespace can be removed from the right-hand side of the string using **`rstrip()`**
- The newline is considered "white space" and is **stripped**

```
>>> input = open('mbox.txt', 'r')
>>> for line in input:
      line = line.rstrip()
      if re.findall('From:',line):
            print(line)
```

```
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
From: gsilver@umich.edu
From: gsilver@umich.edu
From: zqian@umich.edu
From: gsilver@umich.edu
From: wagnermr@iupui.edu
```

A copy of the string is returned with trailing characters removed

Searching: fixed (2)

- The extra return could also have been removed with `line[:-1]`

```
>>> input = open('mbox.txt', 'r')
>>> for line in input:
      line = line.rstrip()
      if re.findall('From:',line):
            print(line[:-1])
```

```
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
From: gsilver@umich.edu
From: gsilver@umich.edu
From: zqian@umich.edu
From: gsilver@umich.edu
From: wagnermr@iupui.edu
```

Skipping

- Lines can be conveniently skipped using the **`continue`** statement.

```
>>> input = open('mbox.txt', 'r')
>>> for line in input:
      line = line.rstrip()
      if not re.search('From:',line):
            continue
```

```
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
From: gsilver@umich.edu
From: gsilver@umich.edu
From: zqian@umich.edu
From: gsilver@umich.edu
From: wagnermr@iupui.edu
```

Selecting Lines

- We can look for a string anywhere `in` a `line` as our selection criteria.

```
>>> input = open('mbox.txt', 'r')
>>> for line in input:
      line = line.rstrip()
      if not '@uct.ac.za' in line:
            continue
      print(line)
```

select lines with '@uct.ac.za'

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
X-Authentication-Warning: nakamura.uits.iupui.edu: apache set sender to
stephen.marquard@uct.ac.za using -f
From: stephen.marquard@uct.ac.za
Author: stephen.marquard@uct.ac.za
From david.horwitz@uct.ac.za Fri Jan  4 07:02:32 2008
X-Authentication-Warning: nakamura.uits.iupui.edu: apache set sender to
david.horwitz@uct.ac.za using -f
From: david.horwitz@uct.ac.za
Author: david.horwitz@uct.ac.za
r39753 | david.horwitz@uct.ac.za | 2008-01-04 13:05:51 +0200 (Fri, 04
Jan 2008) | 1 line
From david.horwitz@uct.ac.za Fri Jan  4 06:08:27 2008
```

You can prompt for an input file name

```
>>> file_name = input('Enter the file name: ')
Enter the file name: mbox.txt
>>> input = open(file_name, 'r')
>>> count = 0
>>> for line in input:
      if 'Subject' in line:
            count += 1
>>> print('There were',count,'subject lines in', file_name)
There were 1798 subject lines in mbox.txt
```

Writing Files

- To **write** a file, you have to open it with mode '**w**' as the second parameter.

```
>>> out_file = open('myfile.txt', 'w')
>>> print (out_file)
<_io.TextIOWrapper name='myfile.txt' mode='w' encoding='UTF-8'>
```

- Note that if the file myfile.txt already exists, opening it in write mode clears the old data so be careful!

Writing Files

- To **write** a file, you have to open it with mode '**w**' as the second parameter.

```
>>> out_file = open('myfile.txt', 'w')
>>> print (out_file)
<_io.TextIOWrapper name='myfile.txt' mode='w' encoding='UTF-8'>
```

- Note that if the file myfile.txt already exists, opening it in write mode clears the old data so be careful!
- Files can also be opened for appending (instead of writing) using the argument '**a**'.

```
out_file = open('myfile.txt', 'a')
```

Closing Files

- When you are done reading/writing, you have to `close` the file to make sure that the last bit of data is physically written to the disk.

```
>>> out_file = open('myfile.txt', 'w')
>>> out_file.write('Lecture 7.2\n')
>>> outfile.close()
```

- An alternative solution:

```
>>> with open('mbox.txt', 'r') as infile:
      print(infile.read())
```

Reading, writing and closing Files

- The write method of the file handle object places data into the file.
- The default write mode is text for reading and writing strings.

```
>>> out_file = open('myfile.txt', 'w')
>>> line1 = 'Lecture 7.2\n'
>>> line2 = File Operation\n'
>>> out_file.write(line1)
>>> out_file.write(line2)
>>> out_file.close()
>>> in_file = open('myfile.txt', 'r')
>>> for line in in_file:
      print(line[:-1])
Lecture 7.2
File Operation
```

Outline

1. Standard input and standard output
2. File operations
3. pandas

- pandas is an open source Python library which provides high-performance, easy-to-use data structures and analysis tools.

```
import pandas as pd
```

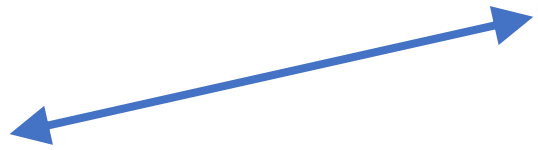
More information can be found at:

http://pandas.pydata.org/pandas-docs/stable/reference/io.html

pandas is useful for reading comma-separated values (csv) files

- Easy method to read these files in a data frame (table).

```
>>> import pandas as pd
>>> data = pd.read_csv("data.txt")
>>> print(data)
    a   b   c   d     name
1   1   2   3   4   python
2   5   6   7   8     java
2   9  10  11  12      c++
pandas.core.frame.DataFrame
>>> type(data)
pandas.core.frame.DataFrame
```



data.txt

```
a,b,c,d,name
1,2,3,4,python
5,6,7,8,java
9,10,11,12,c++
```

pandas can also read Excel files

- Use **`pandas.read_excel()`**

```
>>> import pandas as pd
>>> data = pd.read_csv("data.txt
>>> file_name = 'PCA.xlsx'
>>> df = pd.read(file_name)
>>> print(df)
>>> PCA1 = df['PCA1'].values
>>> PCA2 = df['PCA2'].values
>>> y = df['Group'].values
```

	Sample	Group	PCA1	PCA2
0	N.1	3	97.033298	19.992441
1	N.2	3	69.778300	41.603534
2	N.3	3	78.046376	-39.509308
3	N.4	3	65.435442	9.192919
4	N.5	3	69.859828	-58.749766
5	pre.1	1	-32.996653	97.173964
6	pre.2	1	-58.970954	-27.013138
7	pre.3	1	-53.551257	-11.509871
8	pre.4	1	-48.435232	-9.056897
9	pre.5	1	-52.075736	-4.787465
10	prog.1	2	-14.136827	48.289060
11	prog.3	2	-50.677588	-21.996517
12	prog.4	2	-26.645156	-15.049542
13	prog.5	2	-42.663840	-28.579412

Learning objectives

Now, you should be able to:

- Understand stdin and stdout
- Read to and write from files

浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

Python III: File Operations

Any Questions?

IBI 1, Lecture 7.2

Dr Duncan MacGregor –
duncan.macgregor@ed.ac.uk

Semester 2, 2025/26